CLASS : B. E. (E &TC) **SUBJECT : ESD**

EXPT. NO. : 06 **DATE:**

Aim:

Write TIMER drivers using HAL functions.

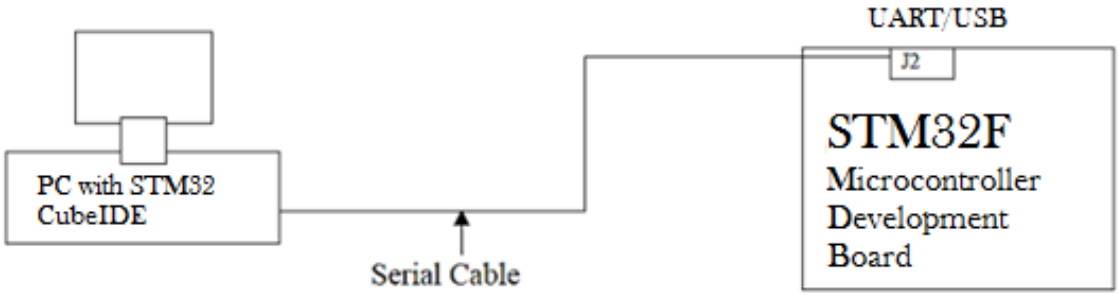
Objectives:

Create project and write code using HAL function to generate delay for timer of STM32F microcontroller.

Requirements:

STM32FXX Microcontroller Development Board, Personal Computer, STM32CubeIDE Software, Connecting wires, Power Supply, etc.

Block Diagram:



Theory:

In this experiment, we will configure the STM32 timer module to operate in the timer mode. And use it to generate a periodic interrupt signal to toggle an LED.

STM32 Timer Mode:

We'll set the overflow time interval to the desired value using the equation down below. And toggle an LED in the interrupt service routine (ISR) for the timer overflow event.

$$T_{OUT} = TicksCount \times TickTime$$

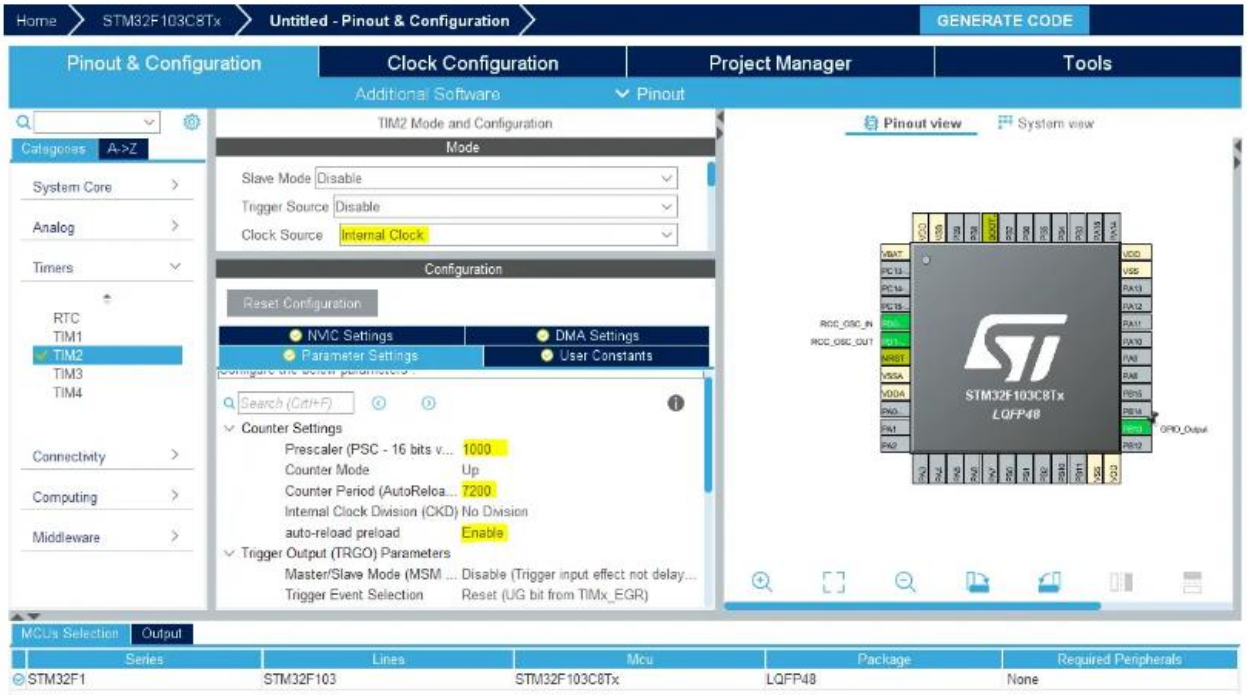
$$T_{OUT} = \frac{PSC \times Preload}{F_{CLK}}$$

the output time interval is determined by the Prescaler value, the clock frequency, and the timer preload

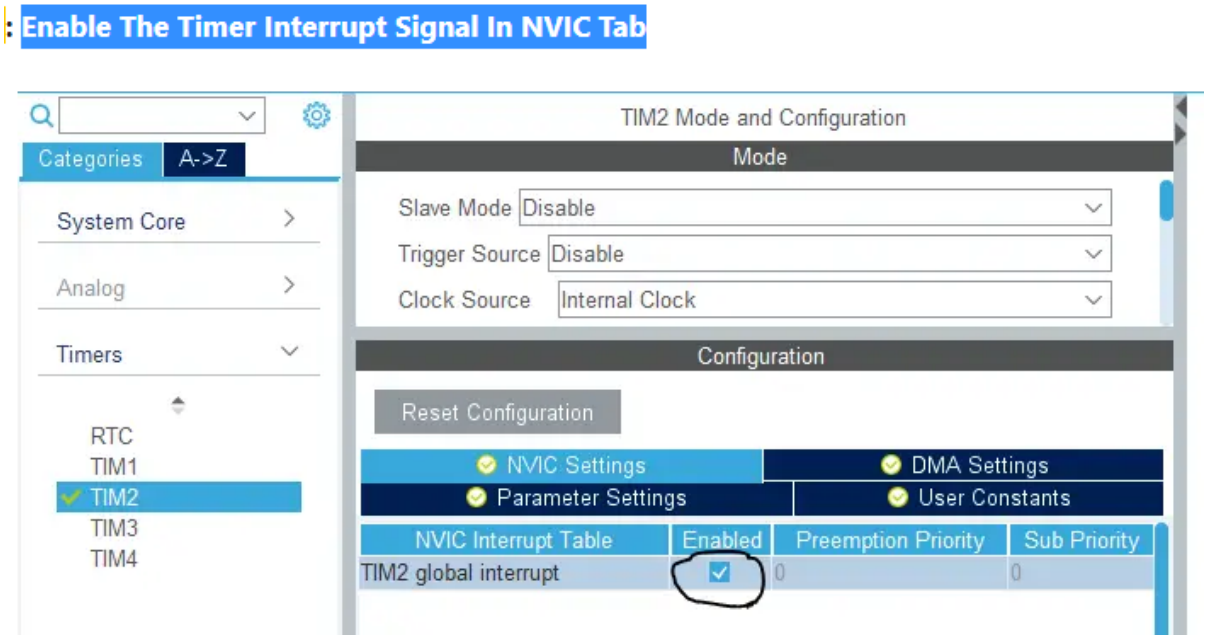
register's value.

Timer3 Peripheral Configuration:

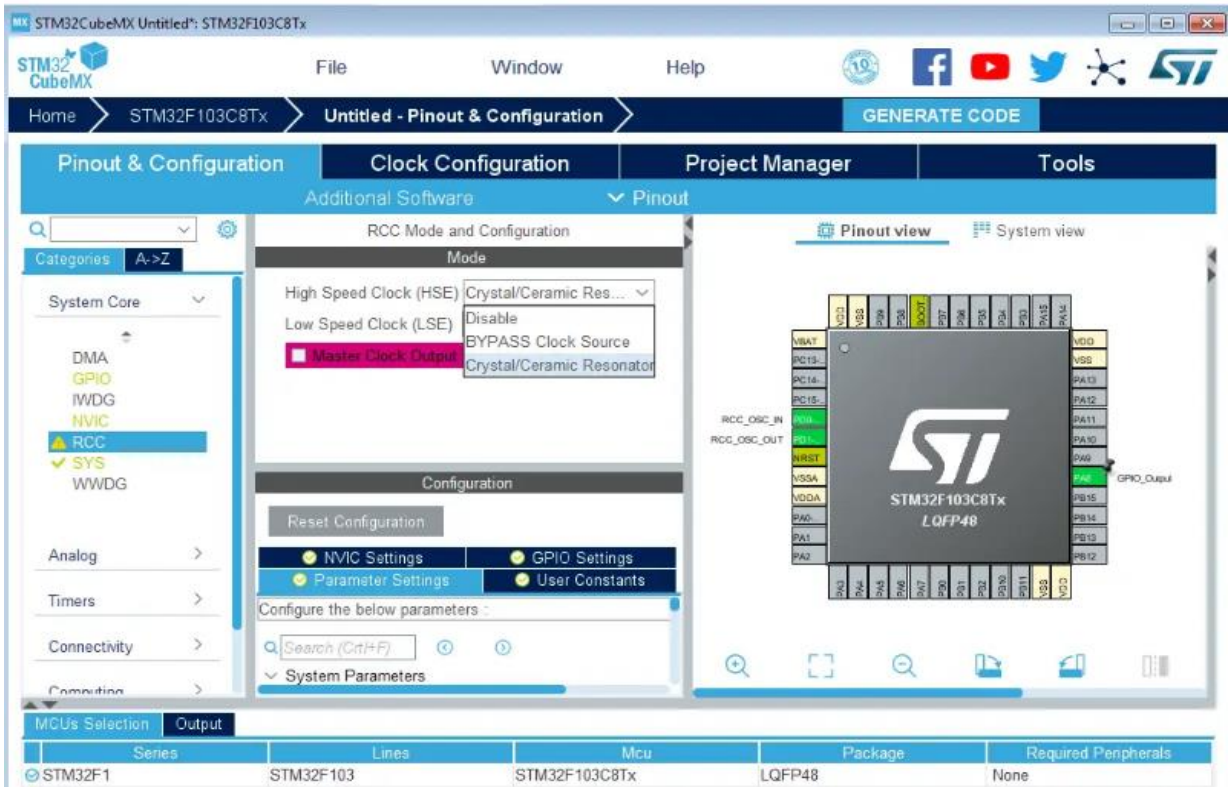
As we've calculated earlier, the Prescaler will be 1000, and the Preload value will be 7200. And the timer module will be clocked at the internal clock frequency.



Enable The Timer Interrupt Signal In NVIC Tab

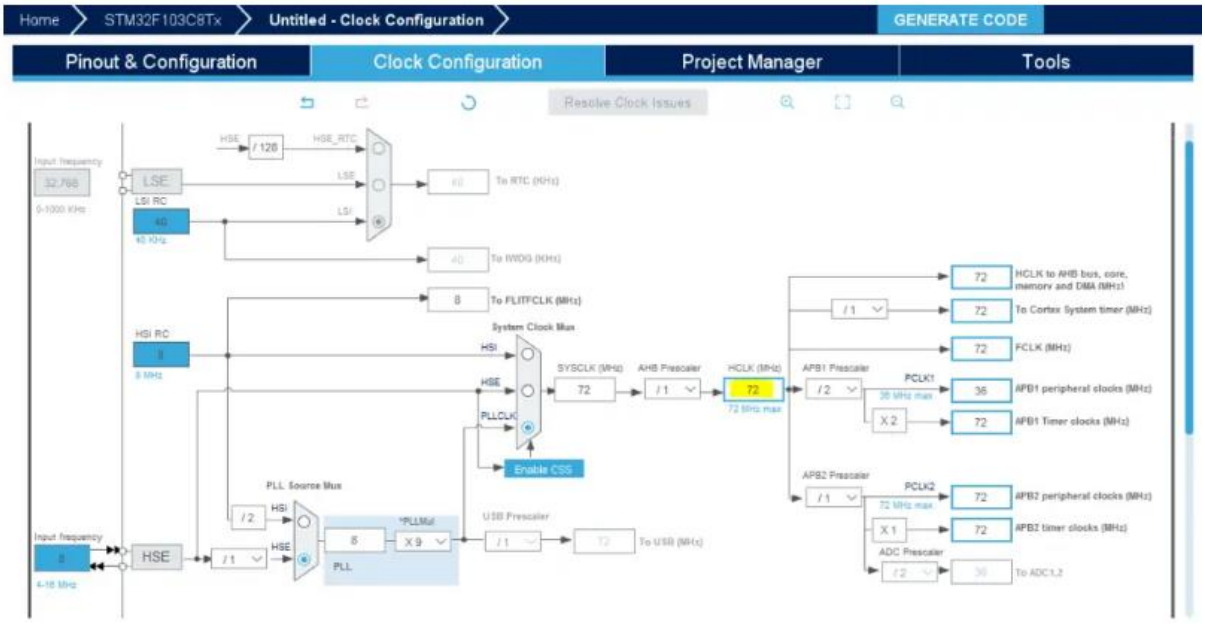


Set The RCC External Clock Source



Go To The Clock Configuration

Set The System Clock To Be 72MHz Or Whatever You Want



Name & Generate The Project Initialization Code For CubeIDE. Then, open the project in the IDE you're using.

And head over to the main.c file. So we can start writing the application code and have a look at the initialization code generated by the STM32 CubeMX tool.

User code for Timer:

```
/* USER CODE BEGIN WHILE */

  while (1)

  {

    /* USER CODE END WHILE */



            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8, 1);

            __HAL_TIM_SET_COUNTER(&htim3, 0);

            while (__HAL_TIM_GET_COUNTER (&htim3) < 250);  // wait for 250ms


            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8, 0);

                    __HAL_TIM_SET_COUNTER(&htim3, 0);

                   while (__HAL_TIM_GET_COUNTER (&htim3) < 500);  // wait for 250ms

    /* USER CODE BEGIN 3 */

  }
```

Building the Project:

Follow the debug and execution procedure mentioned in the previous experiment.

<u>Conclusion:</u>

CLASS : B. E. (E &TC) **SUBJECT : ESD**

EXPT. NO. : 06 **DATE :**

Aim:

Write Analog-to-Digital Converter (ADC) drivers using HAL functions.

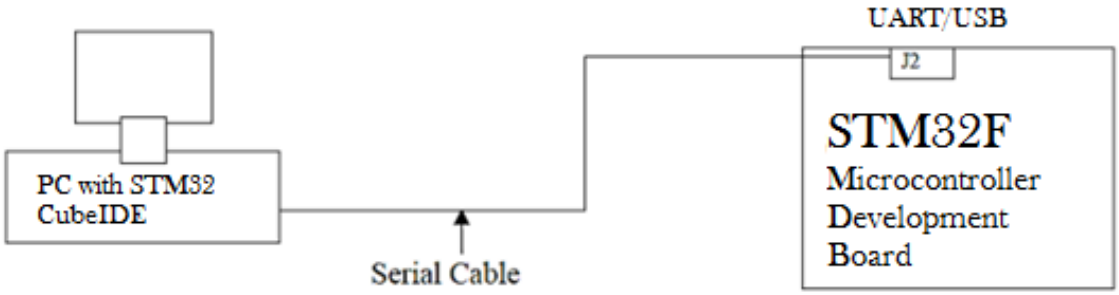
Objectives:

Create project and write code using HAL function to read analog input using inbuilt ADC of STM32F microcontroller.

Requirements:

STM32FXX Microcontroller Development Board, Personal Computer, STM32CubeIDE Software, Connecting wires, Power Supply, etc.

Block Diagram:



Theory:

Analog-To-Digital Converters (ADC):

An ADC (Analog-To-Digital) converter is an electronic circuit that takes in an analog voltage as input and converts it into digital data, a value that represents the voltage level in binary code. The ADC samples the analog input whenever you trigger it to start conversion. And it performs a process called quantization so as to decide on the voltage level and its binary code that gets pushed in the output register.

STM32 ADC:

The STM32F103C8 (Blue Pill) & STM32F432KC have a 12-bit ADC which is a successive approximation analog-to-digital converter. It has up to 18 multiplexed channels allowing it to measure signals from sixteen external and two internal sources. A/D conversion of the various channels can be performed in single,

continuous, scan, or discontinuous mode. The result of the ADC is stored in a left-aligned or right-aligned 16-bit data register.

The analog watchdog feature allows the application to detect if the input voltage goes outside the user-defined high or low thresholds. The ADC input clock is generated from the PCLK2 clock divided by a Prescaler and it must not exceed 14 MHz.

ADC Features:

- 12-bit resolution
- Interrupt generation at End of Conversion, End of Injected conversion and Analog watchdog event
- Single and continuous conversion modes
- Scan mode for automatic conversion of channel 0 to channel 'n'
- Self-calibration
- Data alignment with in-built data coherency
- Channel by channel programmable sampling time
- External trigger option for both regular and injected conversion
- Discontinuous mode
- Dual-mode (on devices with 2 ADCs or more)
- ADC conversion time: 1 µs at 56 MHz (1.17 µs at 72 MHz)
- ADC supply requirement: 2.4 V to 3.6 V
- ADC input range: VREF– ≤ VIN ≤ VREF+
- DMA request generation during regular channel conversion

STM32 ADC Resolution, Reference, Formulas:

- STM32 ADC Resolution:- The STM32 ADC has a resolution of 12-Bit which results in a total conversion time of SamplingTime+12.5 clock cycles.
- ADC Reference Voltage:- The ADC reference voltage pins are defined in the datasheet and assumed to be connected to a voltage level in a certain range.
- STM32 ADC Formulas:
    - ADC Conversion Time

      Tconv = Sampling time + 12.5 cycles
    - ADC Sampling Rate

SamplingRate = 1 / Tconv

- ADC Result Voltage (Analog Input Value)

  Vin = ADC_Res x (Reference Voltage / 4096)

  Where Reference Voltage = (VREF+) – (VREF-)

Different Ways To Read STM32 ADC:

1. The Polling Method
2. The Interrupt Method
3. The DMA Method

Building the Project:

Follow the debug and execution procedure mentioned in the previous experiment.

Conclusion: _____

_____

_____